VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belagavi : 590018

A

Project Report on

"SENSOR-BASED FAULT DETECTION IN ROBOTIC SYSTEM USING TEMPORAL FEATURES"

Submitted in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering

in

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Submitted by

DILSHAD D (1SP22AI025)

Under the guidance of

Sameena Kausar Farheen

Assistant Professor

Dept. of AI & ML

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

S.E.A. COLLEGE OF ENGINEERING & TECHNOLOGY

(Affiliated to Visvesvaraya Technological University, Belgaum)

2025–2026

ABSTRACT

Fault detection in robotic systems has become increasingly crucial due to the growing reliance on autonomous robots in industrial and dynamic environments. Ensuring the reliability and continuous functioning of robotic manipulators is vital to avoid unexpected breakdowns, reduce operational downtime, and maintain safety. However, traditional fault detection techniques often struggle to capture the complex and time-varying patterns that occur in sensor data just before a failure happens.

To address this limitation, this project presents a sensor-based fault detection framework that focuses on extracting and analyzing temporal features from robotic sensor signals. Instead of depending on explicit physical or mechanical models of the robot, the proposed approach utilizes the inherent time-dependent variations in force and torque signals. These temporal features enable the system to differentiate between normal operational behavior and the onset of faulty conditions.

The methodology reduces each six-channel force–torque sample to a single L2-magnitude time series, extracts ten statistical and dynamic features from it, and classifies the result with a Random Forest. The five files of the UCI Robot Execution Failures data overlap heavily: their 463 rows contain only 251 unique samples (65 normal, 186 fault), so duplicates were removed before evaluation to prevent identical samples from appearing in both training and test data. Under repeated stratified five-fold cross-validation the ten-feature model reaches 94.6% accuracy (balanced accuracy 93.0%, macro-F1 0.930), compared with 74.1% for always predicting the majority class and 96.3% for a Random Forest on the 90 raw values. The engineered features therefore provide a compact and interpretable representation whose accuracy is close to, but not above, that of the raw signal. An interactive Streamlit dashboard exposes the predictions, confidence scores and temporal L2 patterns, supporting earlier identification of abnormal events and proactive maintenance.

Table of Contents

Abstract i

List of Figures iv

List of Tables v

1 Introduction 1

2 Literature Survey 3
- 2.1 Introduction . . . 3
- 2.2 Review of Existing Research Works . . . 3
  - 2.2.1 Model-Free Robot Anomaly Detection . . . 3
  - 2.2.2 Autoencoder-Based Network Anomaly Detection . . . 3
  - 2.2.3 Memory-Augmented Autoencoder (MemAE) . . . 4
  - 2.2.4 Graph-Augmented Normalizing Flows for Time-Series Anomaly Detection . . . 4
  - 2.2.5 Subsystem-Based Fault Detection Using L2 Norm and Random Forest 4
  - 2.2.6 Feature Transformation for Robot Execution Failures . . . 4
  - 2.2.7 Random Forests . . . 5
- 2.3 Conclusion . . . 5

3 System Design 6
- 3.1 Requirements . . . 6
- 3.2 Definition . . . 6
- 3.3 System Architecture . . . 7
  - 3.3.1 Data Acquisition Layer . . . 7
  - 3.3.2 Preprocessing Layer . . . 7
  - 3.3.3 Temporal Feature Modelling Module . . . 8
  - 3.3.4 Fault Detection and Classification Module . . . 9
  - 3.3.5 Data Logging and Monitoring Layer . . . 10
- 3.4 Data Flow . . . 10
- 3.5 Algorithm Design . . . 10
  - 3.5.1 Training Phase . . . 11
  - 3.5.2 Detection Phase . . . 11
  - 3.5.3 Algorithm Characteristics and Advantages . . . 12

4 Implementation **13**
4.1 Implementation Overview and Experimental Setup 13
4.2 Dataset Description 14
4.2.1 Structure of the Dataset 14
4.2.2 Overlap Between Files and De-duplication 15
4.2.3 Interpretation of the Data 15
4.2.4 Why This Dataset Fits the Proposed Method 16
4.3 Brief Explanation of the Implementation 16
4.3.1 Data Preparation Script (`prepare_robotexec_files.py`) 16
4.3.2 Training Script (`train_and_save.py`) 16
4.3.3 Dashboard Script (`dashboard.py`) 17

5 Results and Discussion **18**
5.1 Data Preparation Output 18
5.2 Model Training and Evaluation 19
5.2.1 Training Console Output 19
5.2.2 Cross-Validated Performance and Baselines 20
5.3 Dashboard Home Screen 20
5.4 Dashboard Predictions on Example Samples 21
5.4.1 Normal Sample with High Confidence (Row 4) 21
5.4.2 Normal Sample with Lower Confidence (Row 67) 22
5.4.3 Fault with Moderate Confidence (Row 26) 23
5.4.4 Fault with High Confidence (Row 57) 24
5.5 Limitations 25

6 Conclusion and Future Scope **26**
6.1 Conclusion 26
6.2 Future Scope 27

References **28**

List of Figures

4.1 Implemented pipeline 13

5.1 Console output of the data-preparation script 18
5.2 Console output of model training and evaluation 19
5.3 Dashboard home screen 21
5.4 Normal sample, row 4: prediction, L2 plot and feature values 22
5.5 Normal sample, row 67: prediction, L2 plot and feature values 23
5.6 Fault sample, row 26: prediction, L2 plot and feature values 24
5.7 Fault sample, row 57: prediction, L2 plot and feature values 25

List of Tables

3.1 Temporal features extracted from each 15-step L2 series 9

4.1 Experimental setup 14
4.2 Class counts per source file and after de-duplication 15
4.3 First five rows of the cleaned dataset (columns 9–88 omitted for brevity) . . 15

5.1 Classification report on the hold-out test set (51 samples) 20
5.2 Repeated stratified 5-fold cross-validation on the 251 unique samples . . . 20
5.3 Predictions and feature values of the four example samples 21

CHAPTER 1

INTRODUCTION

Robotics has profoundly transformed modern industrial operations by executing tasks with unmatched precision, endurance, and consistency. These robotic systems are now deeply integrated into mission-critical processes, ranging from automated assembly lines to remote operations in hazardous environments such as high-temperature machining, nuclear facilities, and deep-sea exploration. As industries move toward smart manufacturing and full automation, the reliability and operational integrity of robotic systems become not just a performance concern, but a fundamental requirement for safety, productivity, and cost efficiency.

However, this high level of dependency introduces a major challenge: ensuring that robotic systems can detect and respond to faults before they escalate into catastrophic failures. Even minor mechanical or electrical abnormalities, if undetected, can propagate into costly downtime, product defects, or dangerous failures that threaten personnel and property. This makes early, accurate, and automated fault detection a core requirement for modern robotic reliability engineering.

Traditional industrial fault detection methods are still widely used today, including threshold-based alarms, scheduled maintenance inspections, and expert-driven manual diagnostics. Despite being straightforward to apply, these methods suffer from several limitations:

1. **Reactive operation:** They typically recognize faults only after substantial performance degradation or visible symptoms occur, leaving little room for preventive action.
2. **High false-positive rates:** Simple thresholds often cannot differentiate between normal operational fluctuations and true fault conditions, reducing operator trust.
3. **Dependence on expert knowledge:** Effective configuration and interpretation require extensive domain expertise, making deployment and scalability difficult.

With recent technological advancements, machine learning has emerged as a promising solution for predictive fault detection. Data-driven techniques enable systems to autonomously learn fault characteristics from sensor signals such as vibration, current, torque, temperature, position, or velocity [1, 5]. These models can identify subtle patterns that would be invisible using classical statistical techniques. However, practical barriers still remain:

- Many deep learning approaches require large amounts of labeled failure data, which is rarely available in industry because real-world failure events are infrequent.

- Complex algorithms often exceed the computational resources available on standard robotic controllers or embedded systems.
- The interpretability problem persists: "black-box" decisions do not provide actionable diagnostic insights for engineers.

To address these gaps, this research investigates a balanced, hybrid methodology that combines the temporal pattern recognition strengths of machine learning with the transparency and low-complexity benefits of classical signal-processing and statistical approaches. The central idea is that raw sensor data contains valuable temporal signatures—how signals evolve, fluctuate, and correlate over time—that can reveal early indications of developing faults.

Through strategically engineered temporal features and lightweight predictive models, we aim to enable:

- Early identification of abnormal behaviors before they degrade system performance;
- Lower false alarm rates by distinguishing noise from genuine faults;
- Efficient real-time deployment on standard industrial hardware, without requiring high-end GPUs or cloud infrastructure.

Prior work suggests that L2-norm-based feature extraction combined with ensemble learners such as Random Forests can achieve good accuracy while remaining computationally light and relatively interpretable [5, 8]. These approaches capture the advantages of both worlds: the predictive performance of machine learning and the transparency of classical statistical techniques.

Ultimately, this research contributes to the growing domain of predictive maintenance and intelligent robotic systems. It aims to provide a practical pathway between simplistic threshold-based systems and overly complex, data-hungry deep learning models. By developing an explainable, lightweight, and scalable fault detection framework, this work has the potential to enhance robotic reliability and support the next generation of smart, autonomous industrial systems.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

Fault detection in robotic systems has evolved from simple threshold-based strategies to advanced machine learning and deep learning models. With the increasing complexity of robotic platforms and sensor networks, traditional model-based diagnostic approaches are no longer sufficient for handling dynamic, high-dimensional, and time-dependent data. The literature reveals a strong shift toward data-driven methods that utilize temporal, multivariate, and subsystem-specific features for predictive maintenance and anomaly detection. This chapter summarizes the research contributions that shaped the approach adopted in this project, together with the dataset and learning-algorithm references on which the implementation relies.

2.2 Review of Existing Research Works

2.2.1 Model-Free Robot Anomaly Detection

- **Author(s):** Hornung *et al.* (2014) [1]
- **Technique used:** Sensor-based model-free anomaly detection
- **Key findings:** Capable of processing large volumes of sensor and controller data; does not depend on statistical or physical fault models.
- **Limitations:** Poor at detecting diverse and rare fault types; limited interpretability in complex environments.

2.2.2 Autoencoder-Based Network Anomaly Detection

- **Author(s):** Chen *et al.* (2018) [2]
- **Technique used:** Autoencoder-based anomaly detection
- **Key findings:** Learns high-level, nonlinear feature representations; effective for detecting complex anomalies in network-like systems.
- **Limitations:** May reconstruct anomalies as normal samples.

2.2.3 Memory-Augmented Autoencoder (MemAE)

- **Author(s):** Gong *et al.* (2019) [3]
- **Technique used:** Memory-augmented deep autoencoder
- **Key findings:** Uses an external memory to store typical normal patterns; improves anomaly detection precision.
- **Limitations:** Memory addressing is sparse and difficult to optimize; restricted by limited latent-space representation.

2.2.4 Graph-Augmented Normalizing Flows for Time-Series Anomaly Detection

- **Author(s):** Dai and Chen (2022) [4]
- **Technique used:** Graph-augmented normalizing flow (GANF)
- **Key findings:** Integrates Bayesian graphs for better density estimation; strong modeling capability for multivariate sensors.
- **Limitations:** High computational overhead; not suitable for real-time fault detection systems.

2.2.5 Subsystem-Based Fault Detection Using L2 Norm and Random Forest

- **Author(s):** Alamoudi (2024) [5]
- **Technique used:** L2 norm with Random Forest models
- **Key findings:** Reduces the dimensionality of sensor readings; improves subsystem-specific anomaly classification.
- **Limitations:** Limited scalability to multiple robotic subsystems; may require retraining for new platforms.

2.2.6 Feature Transformation for Robot Execution Failures

- **Author(s):** Seabra Lopes and Camarinha-Matos (1998) [6]
- **Technique used:** Feature transformation strategies for learning from force–torque data
- **Relevance:** Introduced the robot execution failure learning problem built from fixed-length force–torque windows, which is the dataset used in this project [7].

2.2.7 Random Forests

- **Author(s):** Breiman (2001) [8]
- **Technique used:** Ensemble of randomized decision trees
- **Relevance:** Provides the classifier adopted in this work, valued for handling nonlinear relationships, robustness to noise, and built-in feature-importance estimates. The implementation uses the scikit-learn library [9].

2.3 Conclusion

The reviewed studies demonstrate a clear evolution in robotic fault detection methodologies. Early model-free systems provided foundational support for anomaly monitoring, while autoencoder-based and flow-based deep learning techniques enhanced detection performance using complex pattern recognition. Recent research has shifted toward temporal and subsystem-based analysis, addressing limitations of data complexity, scalability, and generalization. Despite these advancements, challenges remain in computational overhead, labeled-data requirements, and adaptability to new robotic environments. These gaps motivate the proposed approach that combines temporal feature extraction with efficient machine learning models to deliver a scalable and reliable fault detection framework.

CHAPTER 3

SYSTEM DESIGN

3.1 Requirements

The system must extract temporal features from sensor windows, classify each window as normal or faulty, log predictions and feature values for later analysis, and present results through a dashboard. It should be accurate on noisy data, light enough to run on a standard CPU without a GPU, and modular so that new sensors or models can be added. The implementation uses Python with NumPy and Pandas for data handling, scikit-learn [9] for the Random Forest and evaluation, Joblib for model persistence, Matplotlib for plotting, and Streamlit for the dashboard.

3.2 Definition

The objective of this project is to develop a reliable and scalable fault detection framework for industrial robotic systems by modelling sensor behaviour over time. Modern robotic manipulators operate in complex environments where faults may arise due to mechanical wear, unexpected collisions, excessive loads, sensor degradation, or control errors. Such faults often develop gradually or manifest as subtle temporal deviations rather than abrupt signal failures.

Traditional fault detection approaches that rely on thresholding individual sensor values are insufficient in these scenarios, as they fail to capture evolving signal behaviour and correlations across multiple sensor axes. To overcome these limitations, the proposed system analyzes temporal windows of sensor data and extracts features that describe how the signal evolves during robot motion.

The design follows a subsystem-based approach in which each robotic joint or subsystem is analyzed independently, in line with [5]. This reduces dimensionality, simplifies learning, and improves fault localization. The system integrates signal processing, temporal feature modelling, and supervised machine learning to classify robot states into normal or faulty categories. The architecture supports both offline training and real-time deployment, making it suitable for industrial monitoring and predictive maintenance applications.

3.3 System Architecture

The proposed system architecture follows a layered design in which data flows sequentially through well-defined processing stages. Each layer is responsible for a specific transformation of the data, ensuring separation of concerns and ease of maintenance. The architecture enables consistent processing during both training and deployment phases. The system consists of five major layers:

1. Data Acquisition Layer
2. Preprocessing Layer
3. Temporal Feature Modelling Layer
4. Fault Detection and Classification Layer
5. Data Logging and Monitoring Layer

3.3.1 Data Acquisition Layer

The data acquisition layer collects raw sensor signals from the robotic system or simulation environment. These signals represent the physical state and dynamic behavior of the robot during operation. Typical sensor inputs include joint forces, torques, positions, velocities, or combinations thereof.

Each sensor reading is sampled at regular time intervals, forming a multivariate time series. In the experimental setup used for this project, each data instance consists of measurements from multiple sensor axes recorded over a fixed number of time steps: force and torque readings along three spatial axes are recorded for 15 consecutive time steps during robot motion.

To reduce complexity and improve interpretability, the design restricts each experiment to a single sensor type per subsystem, so that one joint or subsystem is analyzed using one category of sensor signal at a time. This avoids unnecessary coupling between unrelated subsystems and allows the model to learn fault patterns specific to a given sensor configuration. The output of this layer is a stream of raw, unprocessed sensor data that reflects the physical interaction between the robot and its environment.

3.3.2 Preprocessing Layer

The preprocessing layer prepares the raw sensor data for feature extraction and machine learning. Sensor data collected from real robotic systems often contains noise, missing values, and scale variations that can negatively impact learning performance if not handled appropriately.

The first step involves cleaning the data by removing or correcting invalid entries. Missing values may be handled through interpolation or replacement, while extreme outliers are mitigated to prevent distortion of statistical features. Noise inherent in physical sensors is reduced using normalization and smoothing techniques. Normalization then brings all sensor axes to a comparable numerical scale, which is important because many learning models are sensitive to differences in magnitude across features.

The cleaned and normalized sensor stream is segmented into fixed-length temporal windows. Each window contains a predefined number of consecutive time steps and represents a short execution segment of the robot. Windowing transforms the continuous data stream into discrete analysis units that can be processed independently, allows faults occurring in specific time intervals to be detected, and supports real-time operation by processing windows as they arrive.

3.3.3 Temporal Feature Modelling Module

The temporal feature modelling module is the core analytical component of the system. Its purpose is to convert raw time-series sensor data into a compact set of descriptive features that capture the dynamic behavior of the robot over time.

Each window contains multivariate sensor readings across several axes. Using these raw values directly would result in high-dimensional input and poor generalization. The system therefore computes the L2 norm of the sensor vector at each time step:

$$\ell_t = \|\mathbf{x}_t\|_2 = \sqrt{F_x^2 + F_y^2 + F_z^2 + T_x^2 + T_y^2 + T_z^2}, \qquad t = 1, \ldots, 15. \tag{3.1}$$

The L2 norm combines multiple axes into a single magnitude representing the overall intensity of force and torque at that instant. Converting multivariate signals into a univariate temporal sequence of L2 magnitudes achieves dimensionality reduction while preserving essential physical information, following the subsystem-based L2 approach of [5]. The representation is also invariant to axis orientation and sign changes, making it robust to variations in coordinate frames.

From the L2 series $\{\ell_t\}$, ten temporal features are extracted (Table 3.1). Measures of central tendency (mean, median) represent the average operating load; dispersion (standard deviation) captures variability and instability; and extreme values (minimum, maximum) reflect peak stresses. In addition to these static statistics, dynamic features describe how the signal changes over time: a linear regression fitted to the L2 sequence gives the slope, which indicates gradual drifts caused by wear or misalignment; first-order differences $\Delta\ell_t = \ell_{t+1} - \ell_t$ detect sudden changes such as impacts or abrupt load variations; and the signal energy

$$E = \sum_{t=1}^{15} \ell_t^2 \tag{3.2}$$

represents cumulative effort during the window. Together, these features form a concise yet expressive representation of temporal behavior.

Table 3.1: Temporal features extracted from each 15-step L2 series

Feature	Description	Captures
`l2_mean`	Mean of ℓ_t	Average load
`l2_std`	Standard deviation of ℓ_t	Variability / instability
`l2_max`	Maximum of ℓ_t	Peak stress
`l2_min`	Minimum of ℓ_t	Lowest load
`l2_median`	Median of ℓ_t	Typical load
`l2_slope`	Slope of linear fit against time index	Gradual drift
`l2_diff_mean`	Mean of first differences	Net direction of change
`l2_diff_max`	Maximum of first differences	Sudden jumps / impacts
`l2_energy`	$\sum_t \ell_t^2$	Cumulative effort
`raw_sum_abs`	Sum of $\lvert x \rvert$ over all 90 raw values	Overall raw activity

3.3.4 Fault Detection and Classification Module

The fault detection and classification module uses the extracted temporal features to determine whether the robot is operating normally or under faulty conditions. A supervised machine learning approach is adopted, with a Random Forest classifier [8] selected as the primary model.

Random Forests are ensemble learning methods that combine multiple decision trees to improve robustness and generalization. They are well suited to this application owing to their ability to handle nonlinear relationships, their resistance to noise, and their inherent feature-importance estimation.

During training, the classifier learns decision boundaries from labelled feature vectors representing both normal and faulty operating conditions. Each feature vector corresponds to one time window of sensor data. The trained model can then classify new feature vectors during deployment. The output of this module is a predicted class label indicating the system state. The design supports binary classification between normal and faulty states, with the potential to extend to multi-class classification for different fault types. Probability estimates generated by the model can also be used to assess prediction confidence and support risk-aware decision-making.

3.3.5 Data Logging and Monitoring Layer

The data logging and monitoring layer ensures traceability, transparency, and usability of the fault detection system. All processed features, predictions, and detected fault events are recorded for offline analysis and auditing. This layer enables engineers to review historical behaviour, evaluate model performance, and refine detection thresholds. It also supports visualization tools and dashboards that display sensor behaviour, temporal L2 patterns, and classification results in real time. By maintaining a structured record of system activity, this layer facilitates predictive maintenance, root-cause analysis, and continuous improvement of the detection framework.

3.4 Data Flow

The system runs in two phases that share the same preprocessing and feature extraction. In the *training phase*, labelled windows are converted to feature vectors and used to train the classifier offline. In the *detection phase*, new windows pass through identical preprocessing and feature extraction and are classified by the stored model; predictions, timestamps and feature values are logged and shown on the dashboard, and a detected fault raises an alert. Using identical steps in both phases keeps the feature distributions consistent. The pipeline is modular, so subsystems, sensors or models can be replaced without redesigning the rest.

3.5 Algorithm Design

The fault detection algorithm is implemented as a two-phase strategy consisting of a training phase and a detection (deployment) phase. This separation lets learning and real-time detection operate independently, improving efficiency, reliability, and scalability. The objective is to identify deviations from normal robotic behaviour by analyzing temporal patterns rather than instantaneous measurements, enabling detection of both abrupt faults and gradually evolving anomalies caused by wear, misalignment, or component degradation.

3.5.1 Training Phase

The training phase builds the fault detection model from historical sensor data collected under known operating conditions. It is performed offline and therefore has no real-time constraints.

1. **Data collection.** Labelled datasets of normal and faulty executions are collected; each sample contains multivariate readings recorded over time.
2. **Windowing.** Data is divided into fixed-length windows, since faults often manifest as changes over time rather than isolated spikes.
3. **L2 transformation.** The L2 norm of the multivariate signal is computed at each time step, giving a compact 15-point L2 time series per window.
4. **Feature extraction.** The ten features of Table 3.1 are computed: statistical features (mean, median, maximum, minimum, standard deviation), a trend feature (regression slope), first-difference features for abrupt changes, and energy-based features for sustained abnormal effort.
5. **Duplicate removal and model training.** Identical samples are removed first, so that copies of one sample cannot appear on both sides of a split. The feature vectors then form the training set for a Random Forest classifier [8]. The model is evaluated with a stratified hold-out split and with repeated stratified five-fold cross-validation, using accuracy, balanced accuracy, macro-F1, the confusion matrix and class-wise precision and recall, and is compared with a raw-value baseline and a majority-class baseline.
6. **Model persistence.** Once performance is satisfactory, the final model is fitted on all unique samples, serialized and stored for deployment.

3.5.2 Detection Phase

In the detection phase the trained model classifies incoming sensor data in near real time. Live data passes through the same preprocessing and windowing steps used in training; any mismatch between these stages would cause feature-distribution shifts and incorrect predictions. For each new window, the L2 series and the same temporal features are computed and fed to the Random Forest, which outputs a normal/faulty label. Predictions are made window by window, which makes the algorithm resilient to transient noise and isolated fluctuations, focusing instead on consistent temporal patterns indicative of faults. When a fault is detected, the event is logged with timestamp and feature values so that alerts, maintenance procedures or further diagnostics can follow. The approach can be extended to multi-class classification to identify specific fault types.

3.5.3 Algorithm Characteristics and Advantages

The algorithm is model-driven yet data-efficient, since it relies on engineered temporal features rather than high-dimensional raw inputs; this reduces computational complexity and suits real-time or embedded applications. Focusing on temporal behaviour makes it capable of detecting evolving faults, and the L2-norm representation provides a unified, physically meaningful summary of multivariate signals. Separating offline training from fast, lightweight inference makes it practical for industrial systems with limited computational resources and strict latency constraints.

CHAPTER 4

IMPLEMENTATION

4.1 Implementation Overview and Experimental Setup

The implementation is a pipeline of three Python scripts, shown in Fig. 4.1: a data-preparation script that builds the clean dataset, a training script that extracts the features, evaluates the model and saves it, and a Streamlit dashboard that loads the saved model for interactive use. Table 4.1 lists the settings used in all experiments.

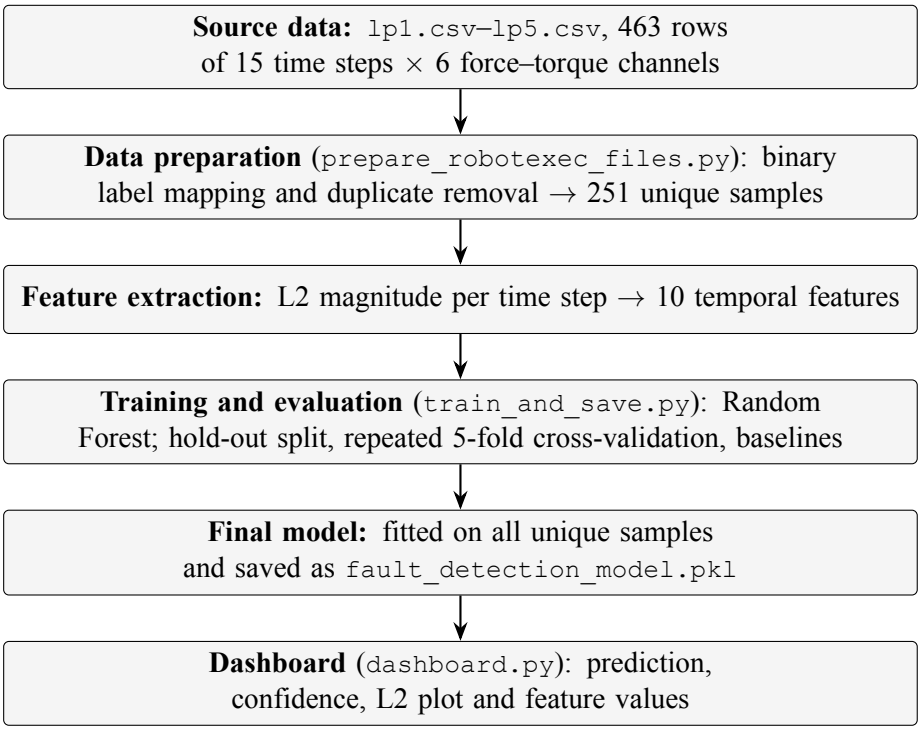


Figure 4.1: Implemented pipeline

Table 4.1: Experimental setup

Item	Setting
Input per sample	15 time steps × 6 channels (90 values)
Data	251 unique samples (65 normal, 186 fault)
Labels	binary: `normal` and `ok` = 0, all other classes = 1
Features	10 temporal features of the L2 series (Table 3.1)
Preprocessing	feature standardization (`StandardScaler`)
Classifier	Random Forest, 100 trees, random seed 42
Hold-out evaluation	stratified 80/20 split, seed 42 (200 training, 51 test samples)
Cross-validation	stratified 5-fold, repeated 5 times with different fold assignments
Baselines	Random Forest on the 90 raw values; majority-class guess
Metrics	accuracy, balanced accuracy, macro-F1, confusion matrix
Software	Python, NumPy, Pandas, scikit-learn, Joblib, Matplotlib, Streamlit

4.2 Dataset Description

The dataset used in this project is the Robot Execution Failures dataset of Seabra Lopes and Camarinha-Matos [6], available from the UCI Machine Learning Repository [7]. It is distributed as five files (`lp1.csv` to `lp5.csv`), each holding repeated executions of a robotic manipulation task in which the force–torque signals were recorded, and each execution carries a class name. The class names differ between files: LP1, LP4 and LP5 use classes such as `normal`, `collision` and `obstruction`, LP2 uses collision directions such as `front_col` and `back_col`, and LP3 uses `ok`, `moved`, `slightly_moved` and `lost`. In this project every class is mapped to a binary label: `normal` and `ok` become 0 (normal) and all other classes become 1 (fault). The five files were combined into a single dataset (`robotexec_merged.csv`) by the data-preparation script.

4.2.1 Structure of the Dataset

Each execution sample captures 15 time steps (frames) of sensor readings. At each time step the robot provides six measurements: F_x, F_y, F_z (force along the X, Y, Z directions) and T_x, T_y, T_z (torque around the X, Y, Z axes). The total number of sensor values per sample is therefore

$$15 \text{ timesteps} \times 6 \text{ channels} = 90 \text{ columns}.$$

Each row of `robotexec_merged.csv` represents a complete 90-dimensional snapshot of the robot's force–torque behaviour during one execution, followed by a `label` column (0 for normal, 1 for fault). Together the five source files contain 463 rows, matching the instance count reported for the repository dataset [7].

4.2.2 Overlap Between Files and De-duplication

The five files are not independent: identical force–torque windows occur in more than one file. All 116 distinct samples of LP4 also appear in LP5, and LP2 and LP3 share 46 samples (carrying different class names in the two files), 45 of which are also contained in LP5. In total, the 463 rows contain only 251 unique samples. If the duplicates are kept, a random train/test split places copies of the same sample on both sides, which inflates the measured accuracy. The data-preparation script therefore keeps one copy of each sample and stops with an error if identical samples ever carry different labels. Table 4.2 lists the class counts per file and after de-duplication.

Table 4.2: Class counts per source file and after de-duplication

File	Rows	Normal (0)	Fault (1)
LP1	88	21	67
LP2	47	20	27
LP3	47	20	27
LP4	117	24	93
LP5	164	44	120
Total (all files)	463	129	334
After de-duplication	251	65	186

Table 4.3: First five rows of the cleaned dataset (columns 9–88 omitted for brevity)

col1	col2	col3	col4	col5	col6	col7	col8	…	col89	col90	label
-1	-1	63	-3	-1	0	0	0	…	-1	0	0
-1	-1	63	-2	-1	0	-1	-1	…	-4	0	0
-1	0	57	-5	-3	0	0	-3	…	-3	0	0
0	-1	59	-2	-1	-1	0	-3	…	-2	0	0
0	-2	65	-4	-2	0	-1	-2	…	-4	-1	0

4.2.3 Interpretation of the Data

Each row corresponds to one complete robotic interaction represented through 15 sequential force–torque measurements, capturing the intensity of contact forces, the dynamics of motion, and the mechanical behaviour of the robot during execution. Force spikes, torque irregularities, or unstable patterns reflect fault conditions such as collisions, obstructions, object displacement, and execution failures. The dataset therefore captures both temporal and dynamic characteristics of robotic movement, making it well suited to time-series feature engineering.

4.2.4 Why This Dataset Fits the Proposed Method

The structure of the dataset (fixed-length, multi-channel, sequential) aligns closely with the proposed method:

- the L2 norm per time step reduces the six-dimensional raw reading into a single magnitude;
- the 15 temporal L2 values summarize the entire execution;
- temporal features (mean, slope, spread, energy, differential spikes) capture behavioural deviations;
- the Random Forest maps these features to normal or faulty states.

4.3 Brief Explanation of the Implementation

The implementation consists of three Python scripts. Their source code is omitted from this report; each script is described below.

4.3.1 Data Preparation Script (`prepare_robotexec_files.py`)

This script combines the five LP files into one clean dataset for machine learning. It reads only the files named `lp*.csv` in the data folder, so outputs of earlier runs are never read again, and extracts the 90 numeric measurement columns of each file. The text class of every row is mapped to a binary label: `normal` and `ok` become 0 and every other class becomes 1. For each file the script prints the class counts and how many rows received each label, which makes a wrong mapping immediately visible. The measurement columns are renamed consistently (`col1`–`col90`) and the files are concatenated. Because the files overlap, the script then removes duplicate samples, keeping one copy of each, and raises an error if two identical samples carry different labels. The result, `robotexec_merged.csv`, contains 251 rows (65 normal, 186 fault) and is ready for training without manual corrections.

4.3.2 Training Script (`train_and_save.py`)

This script implements the training pipeline. It transforms the raw high-dimensional sensor data into temporal features, trains a machine-learning model, evaluates it, and saves it for use by the dashboard.

It loads the merged dataset (dropping any remaining duplicate samples as a safeguard), which holds 90 raw force–torque values per sample plus a fault label; each row is reshaped into 15 time steps with six channels. For each time step the six values are converted into a single L2 magnitude, producing a 15-point temporal signal, from which the ten features of Table 3.1 are computed. A scikit-learn [9] pipeline is built from feature standardization followed by a Random Forest classifier [8] with 100 trees. The script reports a stratified 80/20 hold-out evaluation (accuracy, classification report and confusion matrix), followed by repeated stratified five-fold cross-validation (five repeats) of the ten-feature model, of a baseline Random Forest on the 90 raw values, and of a majority-class baseline. Finally, a model is fitted on all unique samples and serialized with Joblib as `fault_detection_model.pkl` for use by the dashboard.

4.3.3 Dashboard Script (`dashboard.py`)

This script implements an interactive fault detection dashboard using Streamlit, serving as the deployment and visualization layer. It loads the trained model once (cached) and stops with a message if the model file is missing.

The dashboard accepts robot execution data in two modes: (1) *CSV upload*, where the user uploads either a single-row CSV with 90 values or a merged dataset and selects a row to analyze; and (2) *manual input*, where the user pastes 90 numeric sensor values for one execution. For each input, the same L2-based feature extraction as in training is applied, and the feature vector is passed to the model to predict normal or faulty. A confidence score (the highest class probability) is displayed, and fault conditions are highlighted with a prominent red warning banner. The dashboard also plots the temporal L2 magnitudes across the 15 time steps and shows a table of the exact feature values used for prediction, improving transparency and interpretability. Overall, the script turns the trained model into a user-friendly monitoring tool, bridging offline training and practical deployment.

CHAPTER 5

RESULTS AND DISCUSSION

This chapter reports the outcome of the data preparation, the cross-validated performance of the proposed method and of two baselines, and the behaviour of the dashboard on four example samples.

5.1 Data Preparation Output

Fig. 5.1 shows the output of the data-preparation script. For each LP file it lists the classes found and how many rows receive label 0 (normal) or label 1 (fault); the `ok` class of LP3 is counted as normal (20 rows). The five files contain 463 rows (129 normal, 334 fault). After duplicate samples are removed, 251 unique samples remain (65 normal, 186 fault), and the result is saved as `robotexec_merged.csv`.

```
code master ? ❱ uv run prepare_robotexec_files.py
Processing lp1.csv ...
  class counts: {'obstruction': 34, 'normal': 21, 'collision': 17, 'fr_collision': 16}
  → label 0 (normal): 21 | label 1 (fault): 67
Processing lp2.csv ...
  class counts: {'normal': 20, 'left_col': 9, 'back_col': 7, 'front_col': 6, 'right_col': 5}
  → label 0 (normal): 20 | label 1 (fault): 27
Processing lp3.csv ...
  class counts: {'ok': 20, 'moved': 15, 'slightly_moved': 9, 'lost': 3}
  → label 0 (normal): 20 | label 1 (fault): 27
Processing lp4.csv ...
  class counts: {'collision': 72, 'normal': 24, 'obstruction': 21}
  → label 0 (normal): 24 | label 1 (fault): 93
Processing lp5.csv ...
  class counts: {'collision_in_part': 47, 'normal': 44, 'collision_in_tool': 26, 'bottom_collision': 26, 'bottom_obstruction': 21}
  → label 0 (normal): 44 | label 1 (fault): 120

Before de-duplication: 463 rows (normal=129, fault=334)
After de-duplication:  251 rows (normal=65, fault=186)                    Saved data/robotexec_merged.csv. Skipped files: []
```

Figure 5.1: Console output of the data-preparation script

5.2 Model Training and Evaluation

5.2.1 Training Console Output

Fig. 5.2 shows the console output of the training script. The script loads the 251 prepared samples, extracts the engineered L2-based features (feature shape 251×10), trains the Random Forest, and evaluates it on a stratified hold-out set of 51 samples (20%). The hold-out accuracy is **94.1%**, and the confusion matrix is $\begin{bmatrix} 12 & 1 \\ 2 & 36 \end{bmatrix}$: of 13 normal samples 12 were classified correctly, and of 38 fault samples 36 were classified correctly. The per-class metrics are given in Table 5.1. Because the hold-out set contains only 13 normal samples, a single split is a noisy estimate (one additional error changes the accuracy by about two percentage points), so the cross-validated results of the next subsection are the more reliable measure.

```
$ uv run train_and_save.py
Loading merged dataset: data/robotexec_merged.csv
Class counts (0=normal, 1=fault): [65, 186]
Extracting features (this may take a little while)...
Feature shape: (251, 10)
Training RandomForest classifier...

Hold-out test set (20%, seed 42) - one split only, treat with caution:
Accuracy: 0.9411764705882353
Classification Report:
              precision    recall  f1-score   support

           0       0.86      0.92      0.89        13
           1       0.97      0.95      0.96        38

    accuracy                           0.94        51
   macro avg       0.92      0.94      0.92        51
weighted avg       0.94      0.94      0.94        51

Confusion Matrix:
[[12  1]
 [ 2 36]]

Repeated stratified 5-fold CV (5 repeats), mean +- std over repeats:
  10 engineered features   acc 0.946+-0.008 | bal.acc 0.930 | macro-F1 0.930
  90 raw values (baseline) acc 0.963+-0.005 | bal.acc 0.956 | macro-F1 0.952
  majority-class guess     acc 0.741 | bal.acc 0.500

Saving final pipeline (trained on all unique samples) to model/fault_detection_model.pkl
Saved. You can now run the dashboard (streamlit) or load the model.
```

Figure 5.2: Console output of model training and evaluation

Table 5.1: Classification report on the hold-out test set (51 samples)

Class	Precision	Recall	F1-score	Support
0 (Normal)	0.86	0.92	0.89	13
1 (Fault)	0.97	0.95	0.96	38
Accuracy		0.94		51
Macro average	0.92	0.94	0.92	51
Weighted average	0.94	0.94	0.94	51

5.2.2 Cross-Validated Performance and Baselines

Table 5.2 compares the proposed ten-feature model with two baselines under repeated stratified five-fold cross-validation (five repeats with different fold assignments, mean $\pm$ standard deviation over the repeats). The first baseline always predicts the majority class; the second is the same Random Forest trained directly on the 90 raw force–torque values.

Table 5.2: Repeated stratified 5-fold cross-validation on the 251 unique samples

Method	Accuracy	Balanced acc.	Macro-F1
Majority-class guess	0.741	0.500	–
10 engineered L2 features (proposed)	0.946 ± 0.008	0.930	0.930
90 raw values (baseline)	0.963 ± 0.005	0.956	0.952

Because 186 of the 251 samples are faults, always predicting "fault" already gives 74.1% accuracy, so accuracy alone is misleading and balanced accuracy and macro-F1 are reported as well. The proposed model reaches 94.6% accuracy and 93.0% balanced accuracy, far above this trivial baseline. A Random Forest on the 90 raw values scores slightly higher (96.3% accuracy), so on this dataset the engineered temporal features do not improve detection accuracy. Their advantage is a nine-fold smaller input (10 instead of 90 values) whose entries correspond to physical quantities an engineer can inspect, such as mean load, peak, drift and energy, as illustrated by the dashboard below. The standard deviations only reflect the variability due to how the folds are drawn from the same 251 samples; they do not capture variability across robots or recording sessions.

5.3 Dashboard Home Screen

Fig. 5.3 displays the home screen of the Streamlit dashboard, launched with `streamlit run dashboard.py`. The left sidebar lets the user choose the input mode, with the default set to "Upload CSV sample", supporting a single-row CSV containing 90 sensor values or a merged dataset file. The main panel shows the application title and a prompt indicating that the system is ready to process uploaded data.

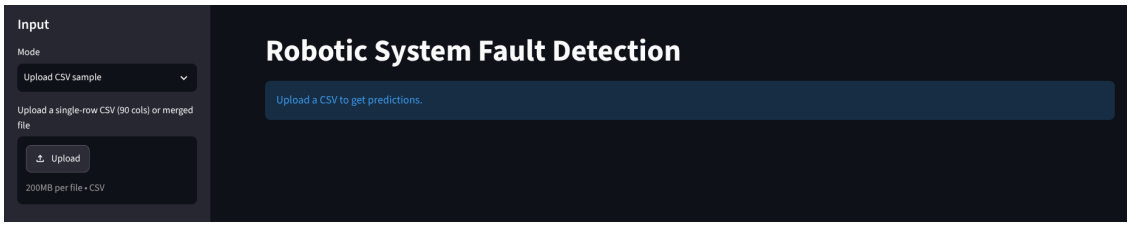


Figure 5.3: Dashboard home screen

5.4 Dashboard Predictions on Example Samples

Four samples of `robotexec_merged.csv` were uploaded to the dashboard: rows 4 and 67 (normal executions) and rows 26 and 57 (faulty executions). Row indices are zero-based positions in the file, and the true class of each sample is stated for reference. Each figure shows the prediction banner, the temporal L2 plot and the feature values used for the prediction. The dashboard model is trained on all 251 unique samples, so these examples illustrate how predictions and features are presented; they are not a held-out evaluation, which is given by the cross-validation above. Table 5.3 lists the predictions and feature values of the four samples side by side.

Table 5.3: Predictions and feature values of the four example samples

	Row 4	**Row 67**	**Row 26**	**Row 57**
True class	normal	normal	collision	obstruction
Prediction	NORMAL	NORMAL	FAULT	FAULT
Confidence	0.96	0.83	0.92	1.00
`l2_mean`	61.2083	57.0449	60.8378	752.3819
`l2_std`	4.7478	2.2548	8.4004	387.3392
`l2_max`	73.6206	60.1249	71.1688	1452.9969
`l2_min`	56.3205	52.5738	34.5688	336.8932
`l2_median`	59.6909	57.1577	60.3490	619.3335
`l2_slope`	0.1739	−0.1155	0.4891	44.4410
`l2_diff_mean`	−0.5224	−0.2775	1.9967	3.2708
`l2_diff_max`	10.4700	6.7390	36.4383	184.5156
`l2_energy`	56535	48888	56577	10741654
`raw_sum_abs`	1103	1202	1305	14876

5.4.1 Normal Sample with High Confidence (Row 4)

Fig. 5.4 shows row 4 (true class: normal), predicted NORMAL with confidence 0.96. The L2 magnitude stays between about 56 and 74 with a single peak at time step 12. The features confirm a stable signal: L2 mean 61.21, standard deviation 4.75, maximum 73.62, slope 0.17 and a largest step change (`l2_diff_max`) of 10.47.

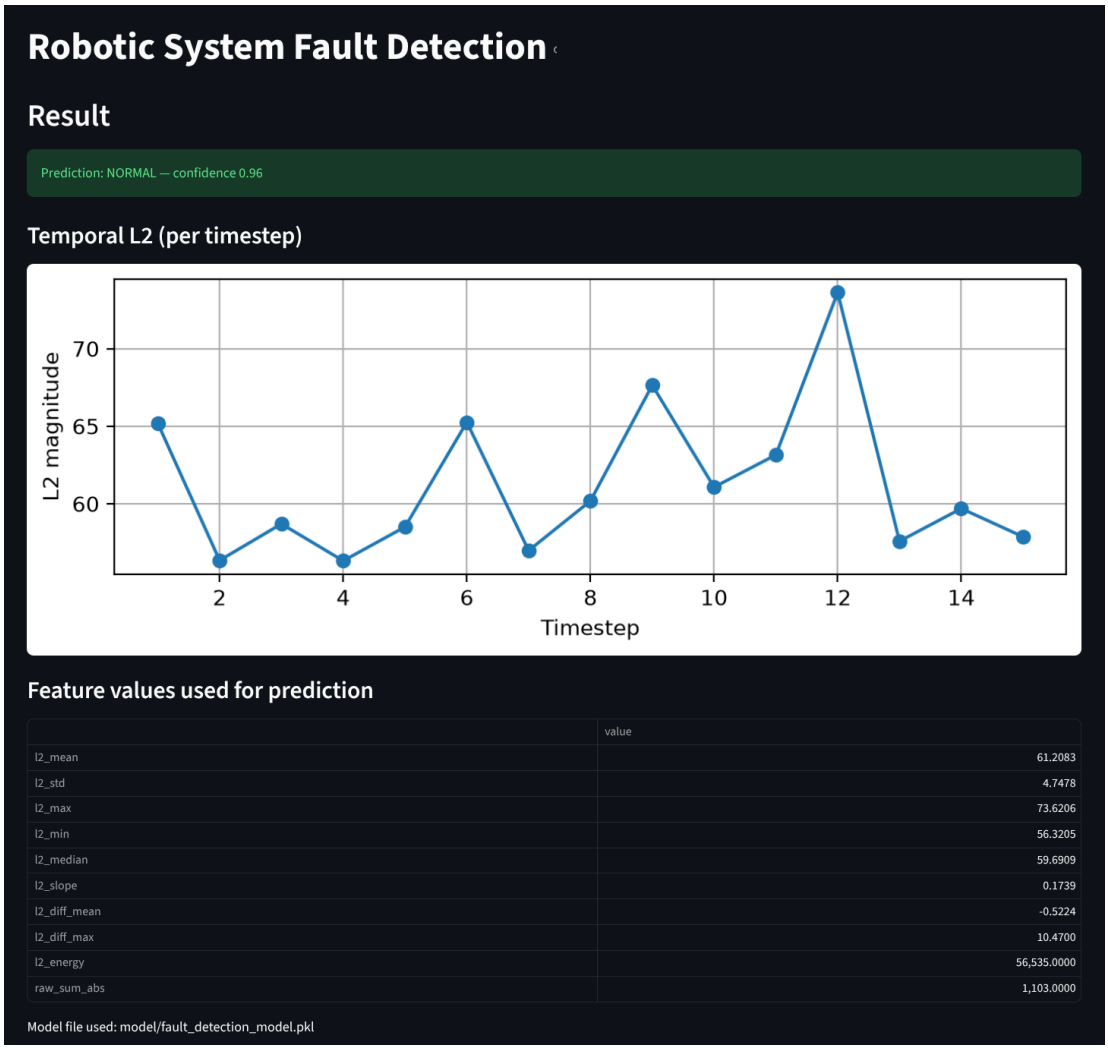


Figure 5.4: Normal sample, row 4: prediction, L2 plot and feature values

5.4.2 Normal Sample with Lower Confidence (Row 67)

Fig. 5.5 shows row 67 (true class: normal), predicted NORMAL with confidence 0.83. The L2 values stay within a narrow band (52.6 to 60.1), with mean 57.04, the smallest standard deviation of the four examples (2.25), a slightly negative slope (-0.12) and a largest step change of 6.74. The reported confidence is the highest class probability estimated by the forest, so a value of 0.83 means that the trees agree less strongly on this sample than on row 4, even though the prediction is correct.

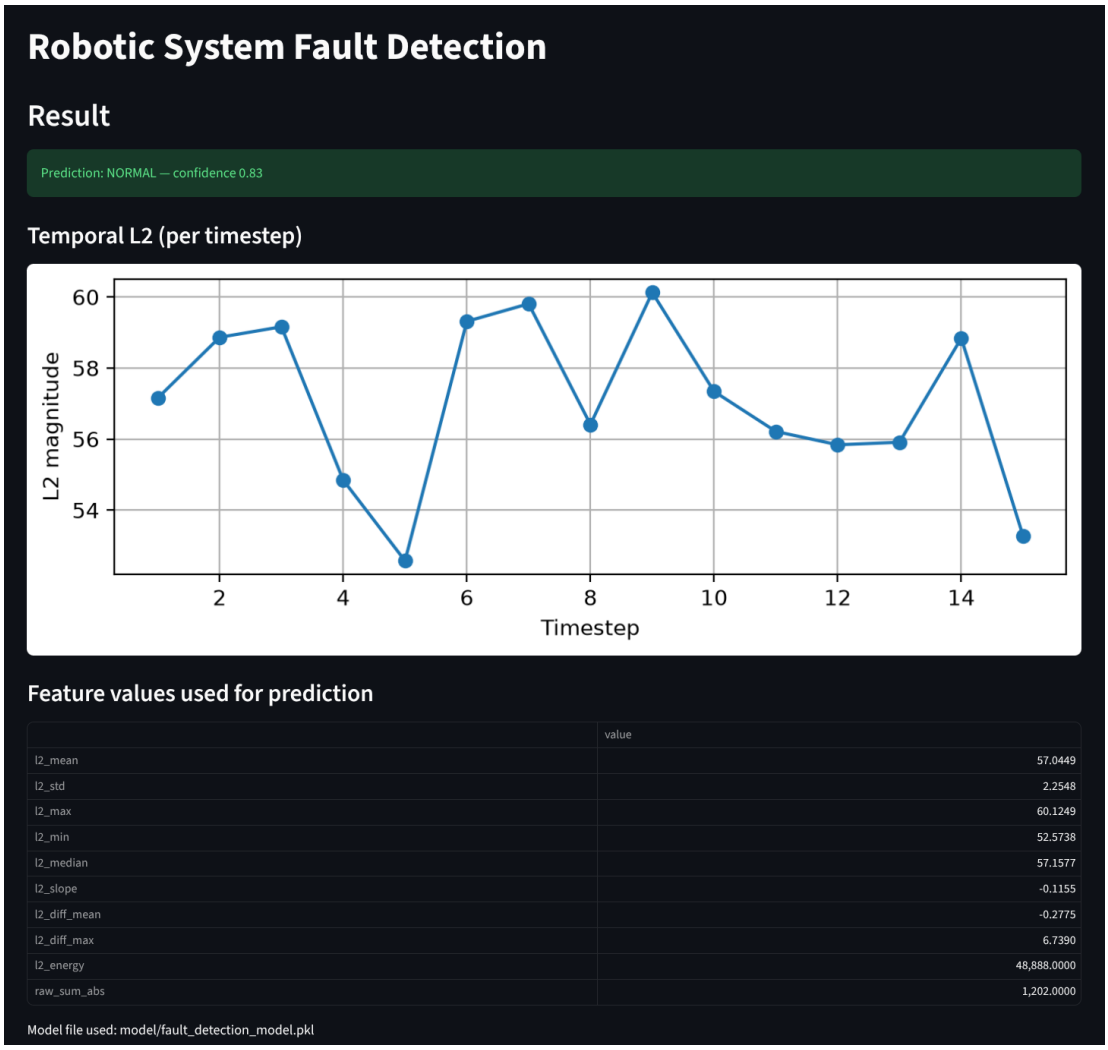


Figure 5.5: Normal sample, row 67: prediction, L2 plot and feature values

5.4.3 Fault with Moderate Confidence (Row 26)

Fig. 5.6 shows row 26 (true class: collision), predicted FAULT with confidence 0.92, displayed with the red alert banner "FAULT DETECTED — ATTENTION REQUIRED". The L2 magnitude starts at 34.6 at the first time step and jumps to about 71 at the next two, before settling around 57–70. The mean (60.84) and energy (56,577) are almost the same as for the normal sample in row 4 (61.21 and 56,535), so these two features alone cannot separate the samples. The fault evidence lies in the variability and the abrupt change: the standard deviation is 8.40 (against 4.75), the minimum is 34.57, and the largest step change is 36.44 (against 10.47).

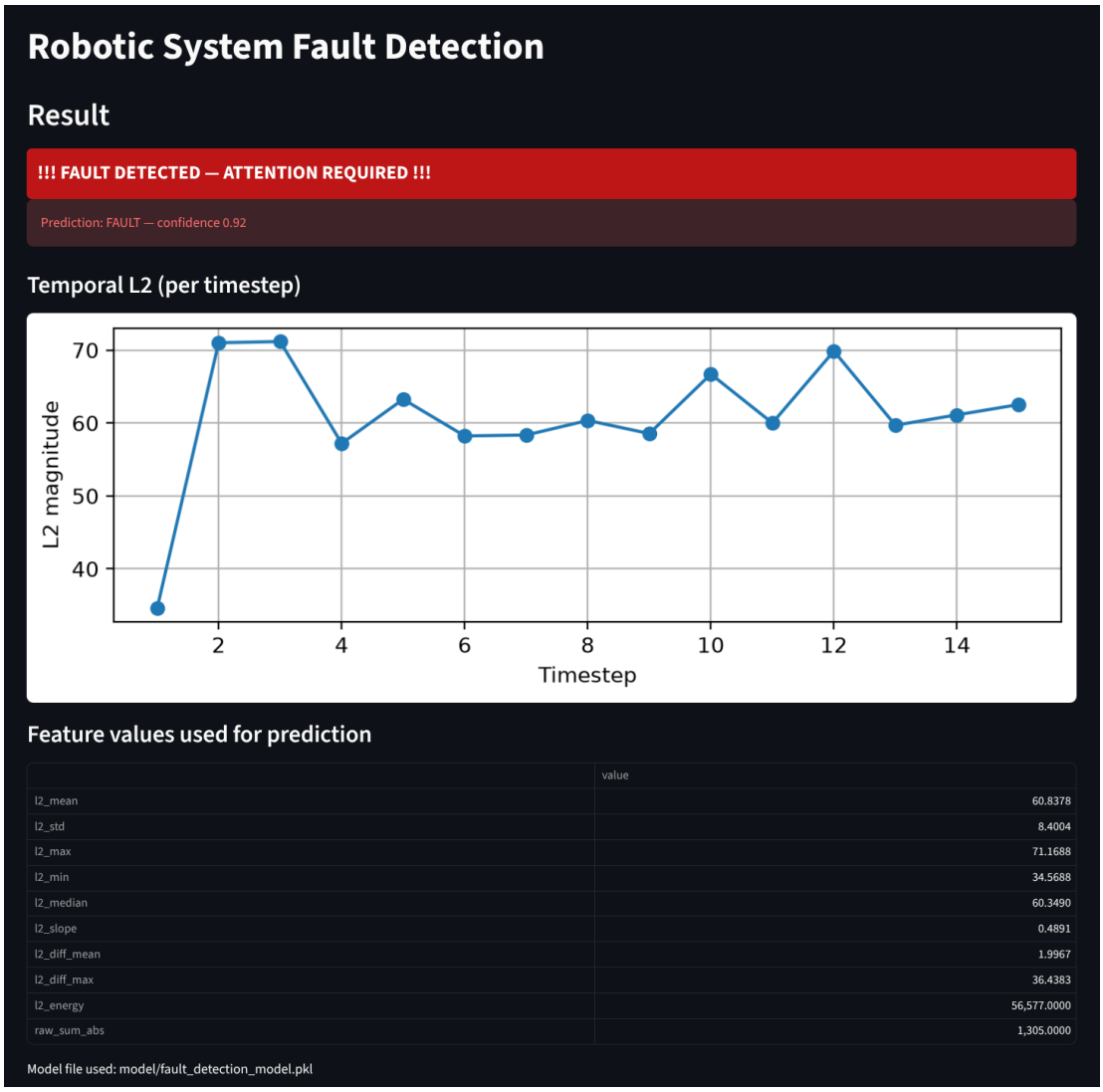


Figure 5.6: Fault sample, row 26: prediction, L2 plot and feature values

5.4.4 Fault with High Confidence (Row 57)

Fig. 5.7 shows row 57 (true class: obstruction), predicted FAULT with confidence 1.00. The L2 magnitude rises steadily from about 337 to about 1,453 at time steps 12 and 13 and then drops back to about 340 at step 14. The features are an order of magnitude above those of the normal samples: L2 mean 752.38, standard deviation 387.34, slope 44.44, largest step change 184.52, L2 energy 10,741,654 (against roughly 5×10^4 for the normal examples) and `raw_sum_abs` 14,876. Such a sustained build-up of force and torque is a clear fault signature.

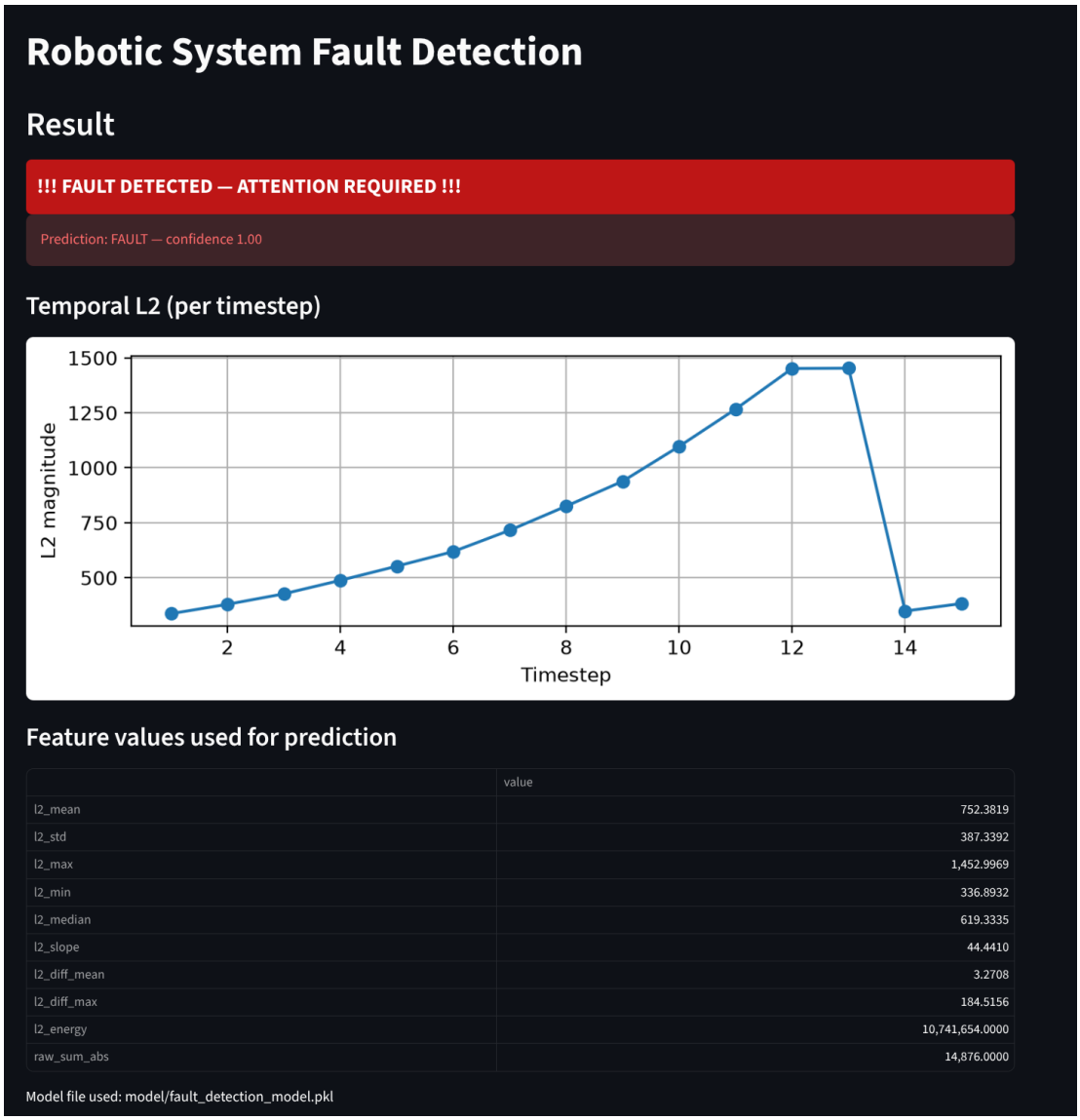


Figure 5.7: Fault sample, row 57: prediction, L2 plot and feature values

5.5 Limitations

The results come from a single small public dataset: after removing duplicates it contains only 251 samples, of which 65 are normal. Samples recorded in the same experimental sessions may still be correlated, so cross-validation can overestimate performance on a new robot or session. All fault types are merged into one class, so the model detects faults but does not identify their kind. Finally, no experiments were run on physical robot hardware.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The proposed "Sensor-Based Fault Detection System Using Temporal Features" extends subsystem-based anomaly detection with an L2-norm and Random Forest foundation [5] by introducing a time-aware analysis method. A purely magnitude-based approach can identify static anomalies but is less able to detect faults that evolve gradually or exhibit temporal irregularities. This project addresses that gap by integrating temporal feature modelling, enabling the system to interpret sequential variations in sensor values rather than evaluating each measurement independently.

Through feature engineering—L2 trend slopes, temporal differences, energy calculations, and statistical descriptors—the system captures behavioural patterns that reflect both normal and faulty robot operation. The five files of the Robot Execution Failures dataset [7] overlap, so duplicate samples were removed, leaving 251 unique samples (65 normal, 186 fault). Under repeated stratified five-fold cross-validation the ten-feature Random Forest reached 94.6% accuracy (93.0% balanced accuracy, macro-F1 0.930), far above the 74.1% obtained by always predicting the majority class. A Random Forest trained on the 90 raw values scored slightly higher (96.3%), so the benefit of the proposed representation lies in its compactness and interpretability rather than in higher accuracy.

The Streamlit interface further demonstrated the system's practicality. When merged sensor samples were uploaded, the dashboard provided instant predictions with confidence scores. Normal samples produced bounded L2 curves with small step changes, whereas faulty samples displayed abrupt jumps, high L2 magnitudes, and elevated energy; for example, the strongest fault example had an L2 energy of about 10.7 million against roughly 5×10^4 for the normal examples. The dynamic visualization and feature breakdown make the model's decisions easy to interpret. The modular design supports adaptation to different robotic platforms, sensor configurations, and subsystem architectures. Note that the results come from a single, small public dataset (65 normal samples) and from cross-validation on samples that may still be correlated through shared recording sessions, so further validation on additional data and on real robot hardware is needed before drawing broader conclusions.

6.2 Future Scope

The results open numerous pathways for further development:

1. **Stronger validation.** Hold out entire experiment files or robot sessions, and collect more data, to estimate how well the model generalizes beyond the samples seen in training.
2. **Advanced temporal learning models.** Sequence models such as LSTMs, GRUs, and Transformers can model long-term dependencies and complex temporal behaviours; replacing or augmenting the hand-crafted features may improve detection of subtle or nonlinear fault patterns.
3. **Real-world deployment and simulation testing.** Implementing the system on real robotic hardware or simulation platforms such as ROS and Gazebo would validate performance under load variations, mechanical wear, environmental disturbances, and multiple fault scenarios.
4. **Finer-grained fault localization.** Future versions could pinpoint the exact joint, actuator, or sensor responsible for an anomaly, moving from fault detection to fault diagnosis.
5. **Multi-sensor fusion.** Integrating encoders, torque sensors, inertial units, current sensors, and vibration modules would help identify faults that manifest across modalities and reduce false positives.
6. **Online, continual, and adaptive learning.** Incremental retraining would let the model adapt to drift caused by wear, lubrication changes, and calibration shifts.
7. **Transfer learning for cross-robot generalization.** Transfer learning and domain adaptation could reduce data-collection overhead and extend the model to different robotic arms or mobile platforms.
8. **Explainable AI.** Frameworks such as SHAP, LIME, or saliency-based temporal analysis would help operators interpret which feature changes triggered an alert.
9. **Multi-agent robotic systems.** Extending the approach to collaborative robots would allow distributed monitoring, coordinated responses, and fault-aware task allocation.

Overall significance. By combining temporal analytics, machine learning, real-time visualization, and a modular architecture, this project provides a foundation for an intelligent fault detection ecosystem that can evolve toward a more autonomous diagnostic tool, contributing to safer, more reliable, and more efficient robotic operations.

References

[1] R. Hornung, H. Urbanek, J. Klodmann, C. Osendorfer, and P. van der Smagt, "Model-free robot anomaly detection," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2014, pp. 3676–3683.

[2] Z. Chen, C. K. Yeo, B. S. Lee, and C. T. Lau, "Autoencoder-based network anomaly detection," in *Wireless Telecommunications Symposium (WTS)*, Phoenix, AZ, USA, Apr. 2018.

[3] D. Gong, L. Liu, V. Le, B. Saha, M. Mansour, S. Venkatesh, and A. van den Hengel, "Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 1705–1714.

[4] E. Dai and J. Chen, "Graph-augmented normalizing flows for anomaly detection of multiple time series," in *Int. Conf. Learning Representations (ICLR)*, 2022.

[5] E. A. Alamoudi, "Subsystem-based fault detection in robotics via L2 norm and random forest models," *IEEE Access*, vol. 12, pp. 167613–167637, Nov. 2024.

[6] L. Seabra Lopes and L. M. Camarinha-Matos, "Feature transformation strategies for a robot learning problem," in *Feature Extraction, Construction and Selection*, 1998.

[7] UCI Machine Learning Repository, "Robot Execution Failures," 1999. [Online]. Available: `https://archive.ics.uci.edu/dataset/138/robot+execution+failures`

[8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[9] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.